

PulseChat - Proje Teknik Dokümantasyonu ve Mimari Tasarım Raporu

Proje Adı: PulseChat (Gerçek Zamanlı Mesajlaşma Sistemi)

Hedef Ölçek: 1.000 – 2.000 Eşzamanlı Aktif Kullanıcı (Concurrent Users)

Geliştirme Metodolojisi: Spec-Driven Development (Spesifikasyon Odaklı Geliştirme) & AI Governance (Agent Denetimi)

1. Yönetici Özeti ve Projenin Amacı

Bu proje, kurumsal standartlara tam uyumlu, modern, yüksek eşzamanlılık kapasitesine sahip ve ölçeklenebilir bir gerçek zamanlı mesajlaşma platformudur. Sistemin öncelikli odağı, 1.000 ila 2.000 aktif kullanıcı arasında milisaniyeler düzeyinde düşük gecikmeli (low latency) anlık mesajlaşma, oda/grup yönetimi, çevrimiçi durum takibi ve asenkron bildirim dağıtımını sağlamaktır. Sistem gereksiz karmaşıklıklardan (over-engineering) arındırılmış, yalnızca açık kaynaklı ve ücretsiz teknolojiler üzerine inşa edilmiştir.

2. Mimari Prensipler ve Tasarım Desenleri (Eksiksiz)

Proje genelinde yüksek sürdürülebilirlik, tip güvenliği, modülerlik ve hatasız mimari kurallar kilitlenmiştir:

2.1. Sistem Mimarileri

- Modular Monolith:** Sistem tek bir çalıştırılabilir (deployable) .NET Core Web API projesi çatısı altında, mantıksal ve fiziksel olarak birbirinden izole modüller (Auth, Chat, Notification, Media) şeklinde kurgulanmıştır. Mikroservis operasyonel maliyetini ve ağ gecikmesini engeller.
- Event-Driven Architecture (EDA):** Mesaj gönderme eylemi HTTP/Socket döngüsünü kilitlemez. Mesaj SignalR Hub'a ulaştığı anda doğrudan RabbitMQ olay kuyruğuna aktarılır; kalıcı veritabanı kaydı ve bildirim dağıtımını arka plan worker'ları tarafından asenkron olarak tüketilir.
- Vertical Slice & CQRS:** Modüller dikey dilimlere (Vertical Slices: SendMessage, GetHistory, MarkAsRead) ayrılmıştır. Komut (Command) ve Sorgu (Query) sorumlulukları kod seviyesinde izole edilmiştir.
- Separation of Concerns (SoC) & Clean Architecture:** Dış katmanlar iç katmanları bilir, ancak çekirdek iş mantığı hiçbir dış aracı (MongoDB, RabbitMQ, SignalR) doğrudan tanımaz; bağımlılıklar arayüzler (Interfaces) üzerinden tersine çevrilir.
- Contract-First & Automated Code Generation (Orval):** Frontend tarafında manuel custom hook, fetch veya axios istekleri yazılmaz. .NET Web API tarafından dışa açılan OpenAPI (Swagger) şeması okunarak, tüm React Query hook'ları ve TypeScript tipleri Orval aracılığıyla otomatik üretilir. İnsan hatası ve tip uyumsuzluğu sıfırlanır.

2.2. Uygulanan Tasarım Desenleri (Design Patterns)

- **Publish-Subscribe (Pub/Sub) Deseni:** RabbitMQ üzerinde kurulan Exchange ve Queue yapısıyla mesajlar birden fazla tüketiciye (MongoDB Worker, FCM Push Worker) kayıpsız dağıtılır.
- **Cache-Aside Deseni:** Kullanıcıların online/offline durumu veya okunmamış mesaj sayacıları önce Redis üzerinde aranır (Cache Hit). Veri yoksa veya ilk kez yükleniyorsa DB'den okunup Redis'e yazılır (Cache Miss).
- **Result Pattern:** API ve Hub yanıtlarında çıplak veri veya doğrudan throw new Exception fırlatmak yerine; isSuccess, data, errorCode ve message alanlarını taşıyan tip güvenli (type-safe) generic bir Result<T> sarmalayıcısı kullanılır.
- **Idempotency Deseni (Mükerrerlik Koruması):** Mobil ağlarda sinyal kopup tekrar bağlandığında istemci aynı mesajı iki kez atabilir. İstemcinin ürettiği benzersiz clientMessageld (GUID), Redis üzerinde kontrol edilerek aynı mesajın çifter çifter veritabanına yazılması engellenir.
- **Centralized Exception Handling & Pipeline Behaviors:** Kodun içine try-catch blokları yığmak yerine, tüm hatalar küresel bir middleware veya pipeline behavior ile yakalanarak tek bir standart JSON formatına dönüştürülür ve merkezi loglama sistemine iletilir.

3. Teknoloji Yığını ve Karar Matrisi

Mimari Katman	Seçilen Teknoloji	Görev ve Sorumluluk	Seçilme Gerekçesi
Backend API	.NET 10 / ASP.NET Core	İş mantığı, WebSocket hub yönetimi, REST API servisleri ve OpenAPI üretimi.	Kurumsal şirket standardı, yüksek thread performansı ve kurumsal mimari desteği.
Canlı İletişim	ASP.NET Core SignalR	WebSockets üzerinden çift yönlü gerçek zamanlı veri akışı.	Ek sunucu maliyeti olmadan oda/grup ve bağlantı yönetimini yerleşik çözmesi.
Önyüz (Web)	Next.js (React / TS)	Web sohbet arayüzü, oturum yönetimi.	Bileşen modülerliği, hızlı render ve modern kullanıcı deneyimi.
Mobil İstemci	React Native	iOS ve Android çapraz platform sohbet istemcisi.	Ortak JS/TS ekosistemi ve yerel cihaz performansı.
API Client & Hooks	Orval (Codegen)	OpenAPI şemasından otomatik	Manuel fetch/hook yazımını bitirir, API

Mimari Katman	Seilen Teknoloji	Grev ve Sorumluluk	Seilme Gerekesi
		React Query hook'ları ve DTO retimi.	deėiřikliklerinde compile-time tip gvenliėi saėlar.
Kod Kalitesi & Format	ESLint & Prettier	Statik kod analizi, linting kuralları ve otomatik kod biimlendirme.	Ekip kodlama standardı, runtime hatalarını nleme ve homojen kod tabanı.
CI/CD & Otomasyon	GitHub Actions	Lint, tip kontrol, build, test kořumu ve sunucuya otomatik deployment.	Hatasız entegrasyon, PR kontrolleri ve gvenli ortam deėiřkeni (Secrets) ynetimi.
Mesaj Veritabanı	MongoDB	Mesaj gemiři, sohbet odaları ve dokman kayıtları.	Aėır yazma (heavy-write) dayanıklılıėı, esnek řema ve bileřik indeksleme.
İliřkisel Veritabanı	PostgreSQL	Kullanıcı kimlikleri, řifre hash'leri ve yetkiler.	ACID garantisi, aık kaynak ve EF Core ile kusursuz uyum.
Mesaj Kuyruėu	RabbitMQ	Asenkron olay daėıtımı (yazma ve bildirim worker'ları).	Gevřek baėlılık (loose coupling), gvenilir teslimat (ACK) ve sıfır kayıp garantisi.
nbellek & Durum	Redis (Docker)	Online/offline durumları, typing bildirimleri ve unread count.	Milisaniyelik bellek ii hız ve veritabanı okuma ykn sıfırlama.
Medya Depolama	MinIO (Self-Hosted)	Grsel, ses kaydı ve dosya saklama (S3 Uyumlu).	cretsiz, aık kaynak nesne depolama, veritabanını řiřirmeme.
Push Bildirim	Firebase (FCM)	Mobil uygulama arka plandayken anlık bildirim iletme.	Tamamen cretsiz, sektr standardı mobil bildirim

Mimari Katman	Seçilen Teknoloji	Görev ve Sorumluluk	Seçilme Gerekçesi
			altyapısı.

4. Depo, CI/CD ve Kalite Güvence Stratejisi

1. Multi-repo Yapısı:

- pulsechat-backend: .NET 10 API, MassTransit/RabbitMQ tüketicileri, EF Core ve MongoDB sürücüsü.
- pulsechat-clients: Monorepo (Turborepo) çatısı altında apps/web (Next.js), apps/mobile (React Native) ve OpenAPI çıktısını barındıran packages/api-client.

2. Statik Kod Kalite Hattı (ESLint & Prettier):

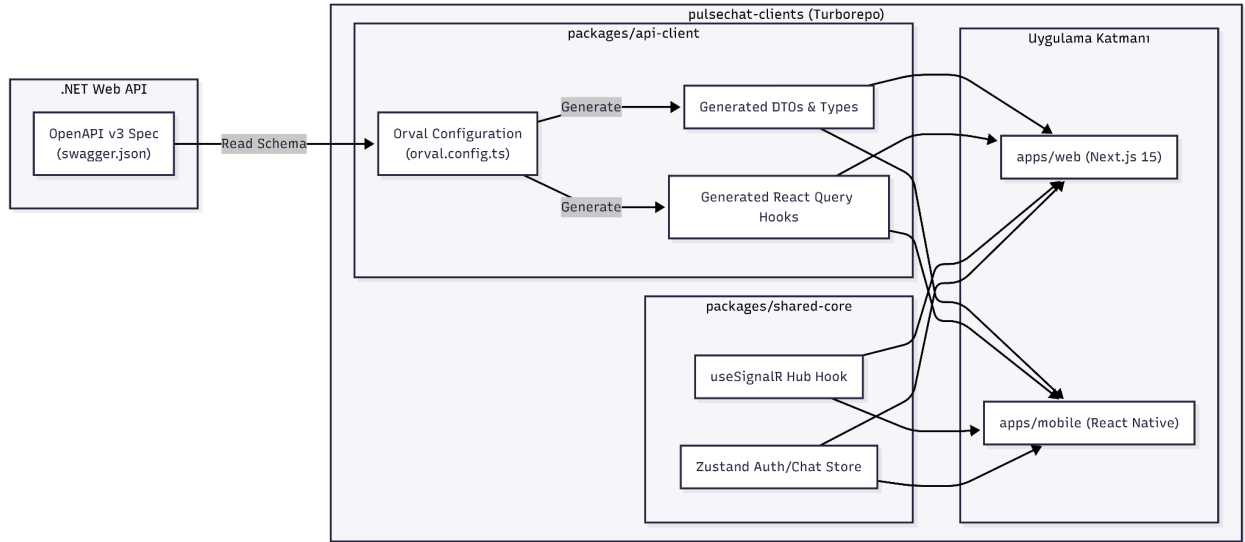
- Her iki istemcide ve backend'de kodlama kuralları kilitlemiştir.
- Hususi kural: Elle tek bir REST API fetch fonksiyonu yazılamaz; Orval tarafından üretilen modeller tüketilir.

3. GitHub Actions Pipeline:

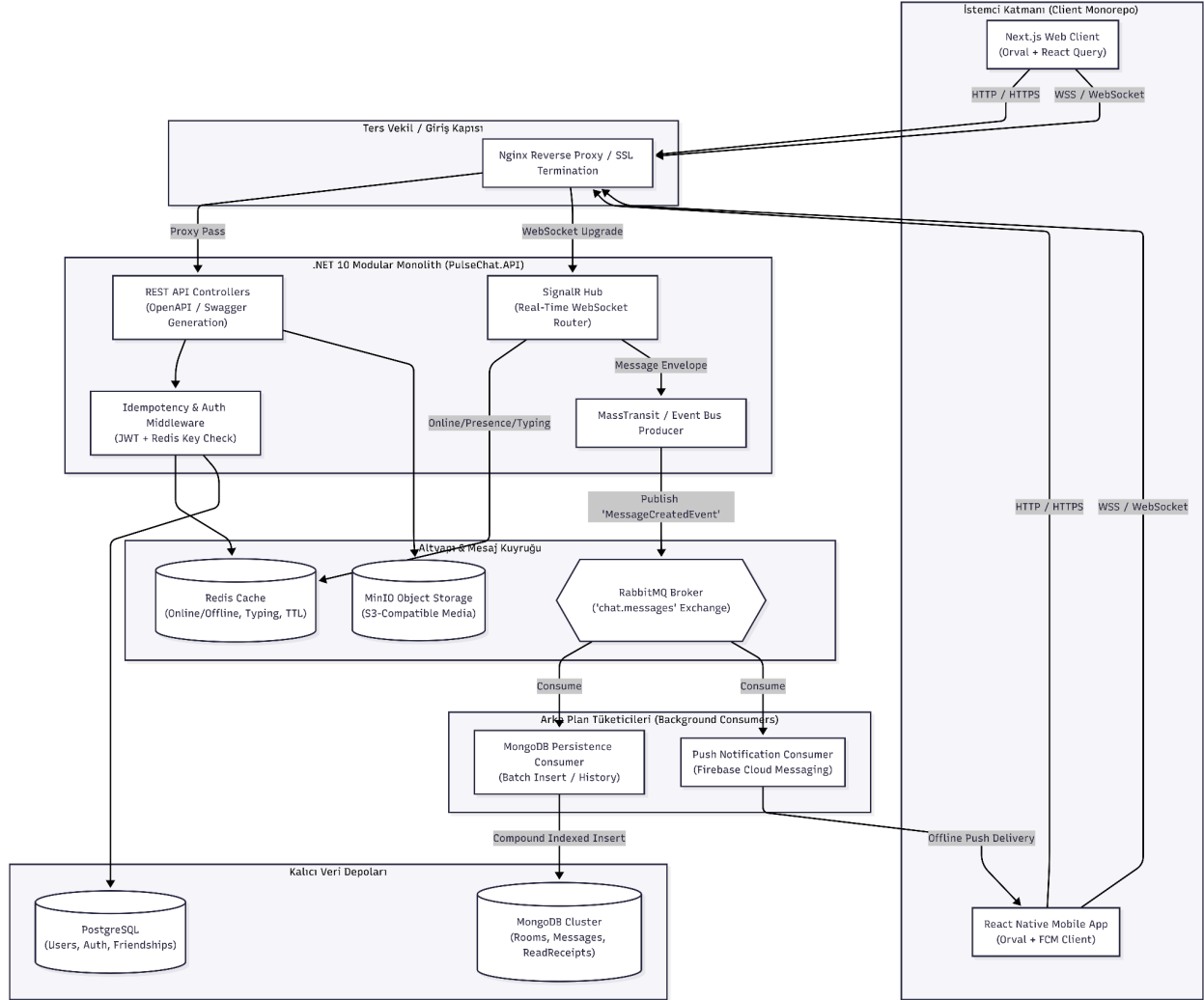
- **Pull Request Aşaması:** ESLint denetimi, Prettier format kontrolü, TypeScript tip doğrulaması (tsc --noEmit) ve Unit Test koşumu. Hata varsa merge kilitletir.
- **Deploy Aşaması:** Docker imajı inşası, GitHub Secrets enjeksiyonu ve prod sunucusuna otomatik servis yenileme.

5. Sistem ve Mimari Diyagramları

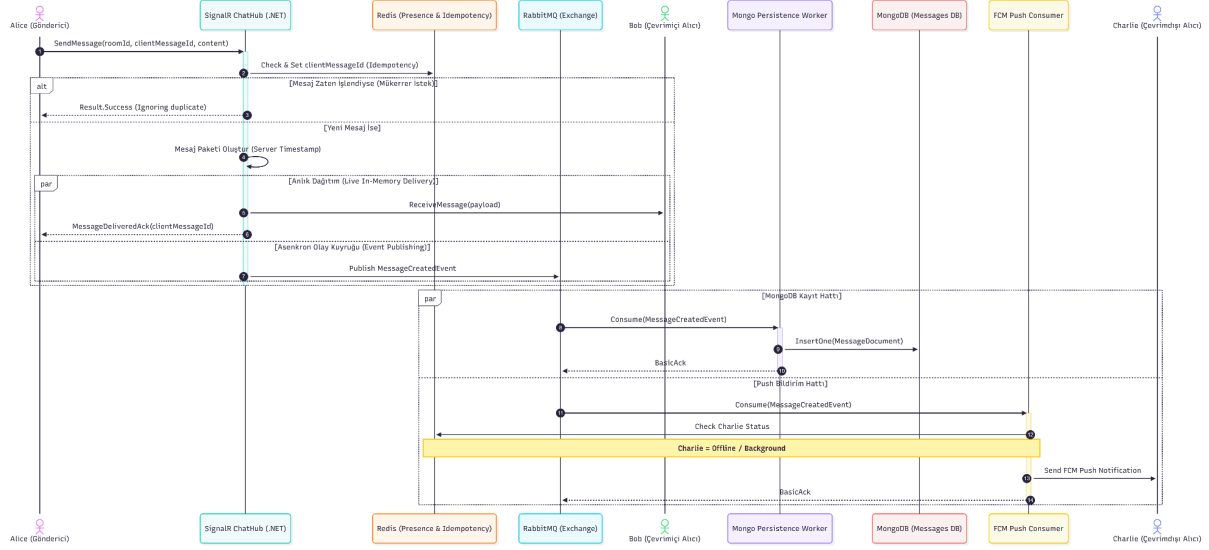
5.1. İstemci Mimarisi Diyagramı (Client Architecture)



5.2. Sistem Mimarisi Diyagramı (System Architecture)



5.3. Mesaj Gönderim Sıralama Diyagramı (Sequence Diagram)



5.4. Veritabanı ve Doküman Şeması (ER & Document Diagram)

